Methods in Deep Reinforcement Learning Used to Play Kirby's Dream Land.

Lucas Hasting

University of North Alabama
Faculty Mentor: Dr. Mark Terwilliger

Florence, Al, United States

Introduction

This paper will look at deep reinforcement learning methods used to play video games, specifically, we attempt to train agents to play against bosses of Kirby's Dream Land. This game was chosen since it is simple for humans to play, short, and is complex enough to study for the purposes of reinforcement learning.

Background

Historical Background

Kirby's Dream Land is an arcade platformer developed by HAL Laboratory and published by Nintendo for the Nintendo Game Boy. The game was proposed and directed by Masahiro Sakurai, a new employee at HAL Laboratory/Nintendo at the time [8]. The main algorithms mentioned are Q-Learning, and Double Q-Learning, which are used with a DQN and a DDQN, described later. The Q-learning algorithm "was first invented in Prof. Watkins' Ph.D thesis "Learning from 'Delayed Rewards"' [3]. Then, a DQN was built in order to try to teach a program to play Atari 2600 games [12]. Finally, the Double Q-Learning algorithm was created by Hado van Hasselt in 2010 to try to solve the overestimation problem that Q-Learning has [6].

Machine Learning

To begin, **machine learning** is "A subfield of artificial intelligence (AI) that gives computers the ability to learn without explicitly being programmed" [1]. We use two subfields of machine learning. They are supervised learning and reinforcement learning. First, **supervised learning** is a subfield of machine learning dedicated to learn a probability distribution $p(y \mid \mathbf{x})$. In other words, we wish to predict the probability of y belonging to a specific outcome using the value of $\mathbf{x}$ (our predictors/input). This comes in two categories of output, classification and regression. **Classification** is where our outcome belongs to a class, and **regression** is where our outcome is some real number on the number line.

We explore a specific subfield of supervised machine learning, referred to as deep learning. Specifically, we are approximating functions using function composition. These relationships are often depicted using a graph which is called a neural network. These models have an input layer (the input vector) and an output layer (the most outside function), where all functions in between are referred to as hidden layers. The simplest form of a Neural Network is a Deep Feedforward Neural Network (DFNN), which only includes linear/activation functions, an example is shown in Figure 1.

When we create Deep Networks, we often use a linear function, and pass that through a non-linear function (to find non-linear relationships), referred to as **activation functions**. We specifically use the ReLU (Rectified Linear Units) activation function, defined below:

$$\mathbf{ReLU}_i(\mathbf{x}) = \max\{0, x_i\}.$$

In our model, we also use convolution layers. A convolution uses a kernel/filter, K, (which acts as a replacement for weight), and an input matrix, I, to perform the discrete convolution operation, defined below:

$$(I * K)_{i,j} = \sum_m \sum_n I_{m,n} K_{i-m,j-n}.$$

Figure 2 shows a visual representation of what the operation is doing. In order to make our models better, we optimize using a loss function. A loss function is a function used to measure how bad of a prediction we made was. We optimize by taking the gradient (a vector pointing towards the slope of the function) of our loss function and adjusting the weights until we get as small as we can, this is called a **local optimum**. Specifically, we use the ADAM (Adaptive Moment Estimation) Optimizer which is defined below with the terms m_t (the moving average of gradients) and v_t (the moving average of squared gradients):

$$W_{t+1} = W_t - \alpha \frac{\frac{m_t}{1-\beta_1^t}}{\sqrt{\frac{v_t}{1-\beta_2^t}} + \epsilon}$$

$$m_t = \beta_1 m_{t-1} + (1-\beta_1)\nabla L(\theta)$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2)(\nabla L(\theta))^2,$$

where $m_0 = 0$, $v_0 = 0$, and ϵ is a really small number used to prevent division by 0. Also, weights are initialized under a Kaiming Uniform distribution which is the distribution $\mathcal{N}(0, \sqrt{\frac{n}{2}})$ [2, 15].

Now, we discuss reinforcement learning. Reinforcement Learning is a subfield in machine learning, which makes use of an agent, that interacts with an environment and learns based on a reward function. A reward function is a function that describes how good the agent was at whatever task it was performing. This approach to machine learning is common with learning how to play games, where an agent acts as a player [4, 5, 7, 14].

Deep Reinforcement Learning

Deep reinforcement learning makes use of both reinforcement learning and deep learning.

0.0.1 Q-Learning. First, we look at the Q-Learning algorithm, which is a reinforcement learning algorithm. The Q in Q-Learning stands for Quality, which determines the quality of a move (this is not the same thing as the reward produced from the reward function). The algorithm builds a table using previous Q values and the current state to estimate the next value of Q after a given action, we will denote this value as Q^*. Initial values of the table are created using random actions, this is referred to as exploring the environment. This algorithmic approach used is referred to as dynamic programming. Q^* is computed using the formula below, which is called the **bellman equation**, with the hyper-parameters α and γ, where s represents state and a represents the action made [14]:

$$Q^* = Q + \alpha R(s, a) + \gamma(\max(Q) - Q).$$

0.0.2 DQN. Next, we look at a Deep Q-Network (DQN). A DQN makes use of two neural networks, and the Q-Learning Algorithm. Instead of a table of Q-values, we store a table of rewards. This table is referred to as the **experience-replay buffer**, in this project we only maintain the 64 most recent entries. The online network makes use of the bellman equation, the experience-replay buffer, and target network to update the weights of the network using this estimation. Both networks have the same architecture, where the input is the current state and the output is every Q-value associated with every possible action. The target network copies the parameters of the online network every N steps, this is done to compute the result of the loss function:

$$L(\theta) = (Q^* - Q(\theta))^2.$$

A visual representing the model's architecture is shown in Figure 3.

0.0.3 DDQN. Moreover, the Q-Learning Algorithm has an overestimation problem with estimating Q-values. The DQN inherits this overestimation problem, this problem is solved with the DDQN (Double Deep Q-Network), which makes use of the Double Q-Learning algorithm. The algorithm modifies the bellman equation in the following way:

$$Q^* = Q + \alpha R(s, a) + \gamma(\hat{Q} - Q),$$

where $\hat{Q}$ is the highest Q value for the action, a ($\max Q \mid a$) [14].

Methodology

For this project, a previous project [10] was referenced to setup Gym Retro. This is the library we use to interact with the emulator gambatte [13]. Then, a tutorial used for a training an agent on Super Mario Bros. (on the Nintendo Entertainment System) using a DQN was followed [16]. The source code in this tutorial was modified by Pedro Alves and Andreas Høiberg Eike from the University of Agder, in order to learn how to play Kirby's Dream Land using a DDQN. However, they stopped after the first mini boss of the game. They also used a different system called PyBoy, so their source code, and reward function, had to be modified to be compatible with Gym Retro [14]. After all the modifications, an agent was then trained on 3 mini bosses and 3 bosses of Kirby's Dream Land, which are shown in Figures 4-10. The model from the first mini boss (Poppy Bros. Sr.) was reused on the mini boss/boss Lololo and Lalala.

The project starts by using 5 wrappers around the environment object created by Gym Retro. These wrappers are defined below:

- **GrayScaleObservation** - Transforms screen data from RGB values to a gray scaled value (lossy compression) to reduce the size of the data.
- **ResizeObservation** - Transforms the gray scaled values into a smaller range.
- **SkipFrame** - Adds an option to skip a specific number of frames per second (determined by a parameter, default is 1).
- **FrameStack** - Stacks screen data into 4 2D arrays (also called a matrix), which will be used for our model [16].
- (Extra) **ResetCompatWrapper** - Used to override the behavior of reset (used to start a new episode) to be compatible with my version of Gym Retro.

We use the screen data to represent our state for the Neural Network, which predicts our Q value. Every frame, Kirby makes an action, stores the reward of that action in the experience-replay buffer, adjusts the weights/kernels of the online network, and every 100 frames, the parameters of the online network get copied to the target network. Every episode (one episode is one attempt at defeating the boss/mini boss), the results are logged so we can determine how well the agent did. The Hyperparameters and reward function used are shown in Table 1 and Algorithm 1 respectively.

Results

Graphs showing the moving average of rewards by episode of each agent is shown in Figures 11-17. We found that the model used for Poppy Bros. Sr. learned how to defeat the boss and it also successfully defeated the mini boss Lololo and Lalala. However, it was only able to reduce the boss Lololo and Lalala to half health. The models for Whispy Woods, Kracro, and King DeDeDe failed to learn how to fight the boss, but did show they were able to hit the boss at least once. The model for Kraco Jr. got stuck after learning that it could make micro movements in the top corner without getting hit, resulting in many small rewards.

Finally, there were two models created for Kabula, the first model performed poorly, it replaced a negative reward of 100 to 10 as an experiment, but an excess in reward allowed the average reward to drop causing the model to perform poorly after it started to perform well. The reward function was tweaked back to the original function and the model performed well overall, and it was able to defeat the boss. A video demonstrating these results can be seen at [11].

Future Work

For future work, better agents can be trained for the bosses that showed poor results. Also, agents can be trained to handle different stage types (horizontal left/right, vertical up/down, enter door, and maze), or each segment of the game individually (the slowest option). A problem for the future will be stage 2, because that stage includes a maze that could be hard for an agent to learn how to get through. Moreover, an API was created in conjunction with a decision tree on data collected prior to this project in order to classify which agent would be used (for using agents trained on specific stage types).

This API had to be used due to Python version compatibility issues with Gym Retro (which requires Python version 3.8). This API could be improved by exploring other models. Overall, the reward function used was mostly effective in playing Kirby's Dream Land. Therefore, Deep Reinforcement Learning using a DDQN was the most effective approach to learning how to play the game. Moreover, these techniques can be applied to learn in other areas as well, including but not limited to, autonomous robotics, data center cooling, finance, and transportation [9].

References

[1] Andriy Onufriyenko. Machine learning, explained. https://mitsloan.mit.edu/ideas-made-to-matter/machine-learning-explained. Accessed: 2026-04-13.

[2] Geekforgeeks. Kaiming initialization in deep learning. https://www.geeksforgeeks.org/deep-learning/kaiming-initialization-in-deep-learning/. Accessed: 2026-04-28.

[3] Geekforgeeks. Q learning ::: The foundation of modern reinforcement learning. https://cml.rhul.ac.uk/qlearning.html. Accessed: 2026-04-28.

[4] Geekforgeeks. What is adam optimizer? https://www.geeksforgeeks.org/deep-learning/adam-optimizer/. Accessed: 2026-04-13.

[5] Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org.

[6] Hasselt, H. Double q-learning, 2010.

[7] IBM. What are convolutional neural networks? https://www.ibm.com/think/topics/convolutional-neural-networks. Accessed: 2026-04-13.

[8] Kirby Wiki. Kirby's dream land. https://kirby.fandom.com/wiki/Kirby%27s_Dream_Land. Accessed: 2026-04-13.

[9] Li, Y. Reinforcement learning applications. *CoRR abs/1908.06973* (2019).

[10] Lucas Hasting. Kirby-dream-land-ai. https://github.com/LucasHasting/Kirby-Dream-Land-AI. Accessed: 2026-04-28.

[11] Lucas Hasting. Kirby's dream land - deep reinforcement learning demo. https://youtu.be/TSWY3JiOhro. Accessed: 2026-04-28.

[12] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., and Hassabis, D. Human-level control through deep reinforcement learning. *Nature 518*, 7540 (Feb. 2015), 529–533.

[13] OpenAI. Gym retro. https://openai.com/index/gym-retro/. Accessed: 2026-04-13.

[14] Pedro Alves†, Andreas Høiberg Eike†. Reinforcement learning with pyboy. https://raw.githubusercontent.com/lixado/PyBoy-RL/main/README/report.pdf. Accessed: 2026-04-13.

[15] PyTorch. Adam. https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html. Accessed: 2026-04-28.

[16] Yuansong Feng, Suraj Subramanian, Howard Wang, Steven Guo. Train a mario-playing rl agent. https://docs.pytorch.org/tutorials/intermediate/mario_rl_tutorial.html. Accessed: 2026-04-13.

Figures, Tables, and Algorithms

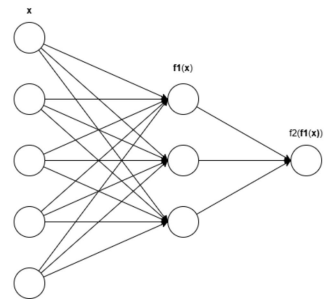

Figure 1: DFNN Example

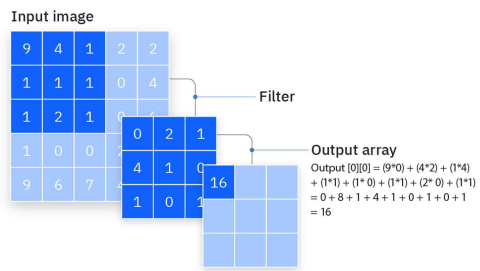

Figure 2: Convolution Visual

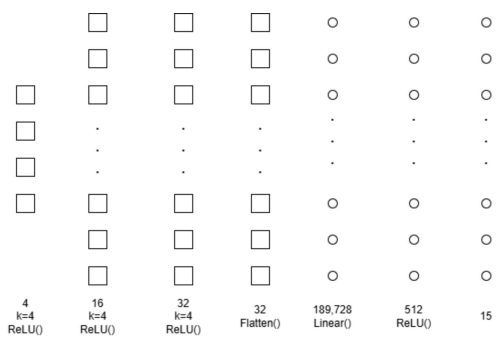

Figure 3: Model Architecture

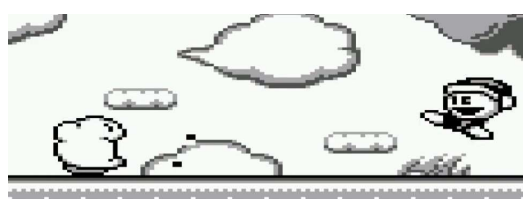

Figure 4: Poppy Bros. Sr.

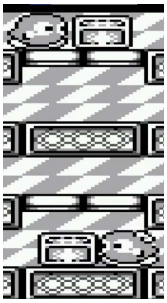

Figure 5: Lololo, Lalala

Figure 6: Whispy Woods

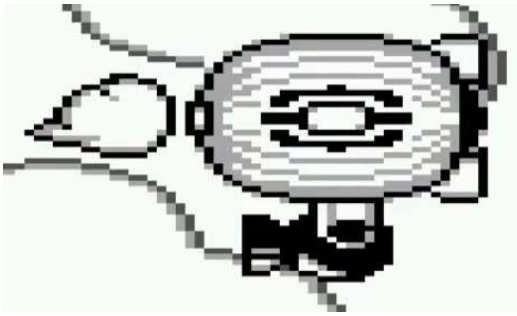

Figure 7: Kabula

Figure 8: Kraco Jr.

Figure 9: Kraco

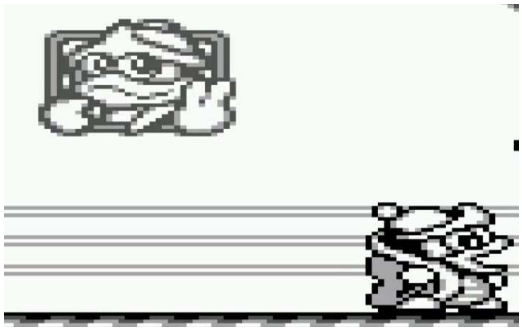

Figure 10: King DeDeDe

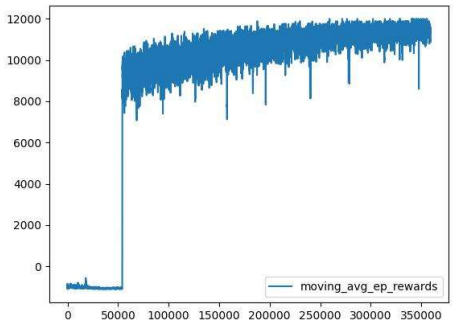

Figure 11: Results - Poppy Bros. Sr., Lololo, Lalala

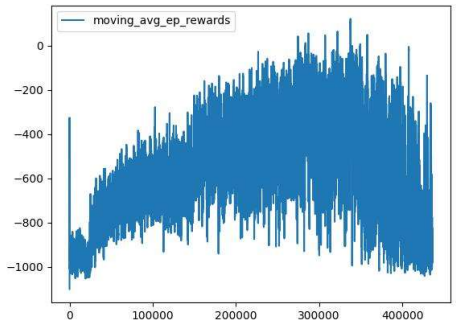

Figure 12: Results - Whispy Woods

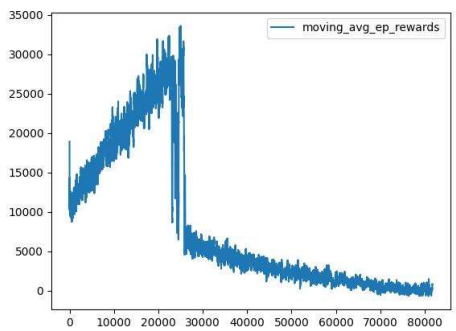

Figure 13: Results - Kabula (Bad Attempt)

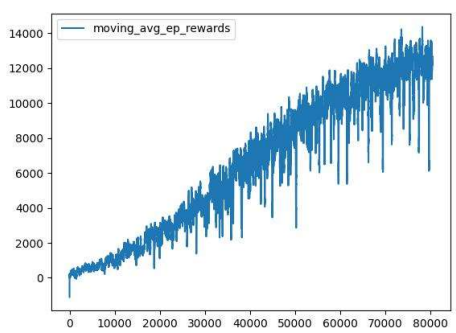

Figure 14: Results - Kabula (Good Attempt)

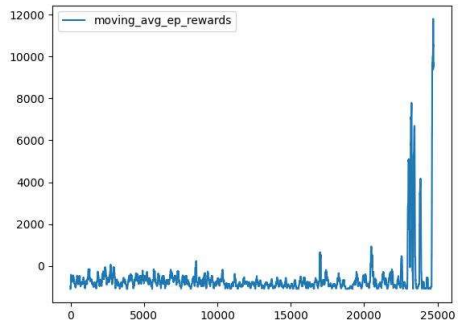

Figure 15: Results - Kraco Jr.

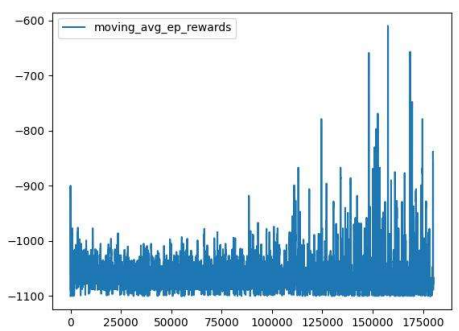

Figure 16: Results - Kraco

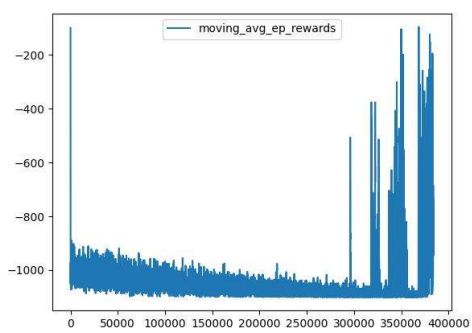

Figure 17: Results - King DeDeDe